PES University, Bengaluru

(Established under Karnataka Act 16 of 2013)

END SEMESTER ASSESSMENT (ESA) - Jan - May 2024

UE21CS341B - Compiler Design

Total Marks : 100.0

1.a. i) Explain in brief the role of a Lexer. Is Lexer implementation language dependent? Justify your answer with at least 2 simple examples. **(04-Marks)**

ii) Why Input buffering is used in lexical analysis? What are the commonly used buffering methods? Explain in brief the advantage of having sentinels at the end of each buffer halves in buffer pairs? **(04-Marks)**

iii) Mention the methods to separate the keywords(aka reserved-words) and identifiers of the language when the scanner reads the lexemes of those during lexical analysis. **(02-Marks)** (10.0 Marks)

1.b. i) Using the following patterns explain the design of a lexical analyzer. Show the working of your lexical analyzer that simulates an NFA on the input string: abaabb

```
ab(b)?      {// some action ....}
aab         {// some action ....}
aba         {// some action ....}
```

(04-Marks)

ii)Explain in brief Lex tool. Consider the following lexical specification

```
%%
aa          printf("No shortcut")
b?a+b?      printf(" to ")
b?a*b?      printf("Success")
```

Given the following input string: bbbaabb
What does the lexer print?
(02-Marks)

iii) Explain with an example Panic-Mode and Phrase-Level (or Statement Mode) Error Recovery Strategies.
(04-Marks) (10.0 Marks)

1.c. Answer the following
i) Eliminate left recursion from the below grammar **(03-Marks)**
A → Bxy | x
B → CD
C → A | c
D → d

Note: The grammar has indirect left recursion

ii) Left Factor (Eliminate common prefixes) the below grammar **(02-Marks)**
A → aAB | aBc | aAc (5.0 Marks)

2.a. i) For the following grammar answer the following:

S → a | ^ | (L)
L → SA
A → ,SA | ϵ

- Construct first and follow table. **[02-Marks]**
- Construct LL(1) table and Mention whether the grammar is in LL(1) or not. **[02-Marks]**
- If the grammar is in LL(1), parse the string: (*a*,*a*) **[03-Marks]**

ii) Explain in brief Recursive-Descent Parsing (RDP). **[02-Marks]** (9.0 Marks)

2.b. Consider the following grammar:
S → SS+ | SS* | a

- Construct LR(0) automata. **(03-Marks)**
- Construct SLR parsing table. Is the grammar SLR? **(02-Marks)**
- Show how to parse the input *aa+* using SLR parser **(03-Marks)**

(8.0 Marks)

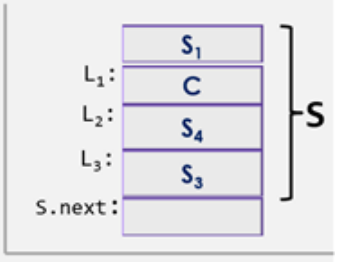
2.c. i) Explain in brief Canonical LR(1) Items. **(02-Marks)**
ii) Why is LR(1)/CLR(1) parser so powerful? **(02-Marks)**
iii) Explain in brief Inherited Attribute and L-attributed SDT. **(04-Marks)** (8.0 Marks)

3.a. (i) Consider the following Context Free Grammar for variable declaration in C language:
D → TL
T → int | float | char | double
L → L_1, id | id

Write a Syntax-Directed Definition (SDD) to add/update type information (i.e., int/float/char/double) of the variables in declaration statement to the symbol table during parsing. **(04-Marks)**

(ii) Consider the following SDD (to generate intermediate code) for for-statement

Production	Semantic Rules
S -> for(S_1; C; S_3)S_4	L1=new(); L2=new(); L3=new(); S_1.next=L1; C.true=L2; C.false=S.next; S_4.next=L3; S_3.next=L1; S.code=S_1.code||label||L1||C.code||label||L2||S_4.code ||label||L3||S_3.code



Let us turn this L-attributed SDD into an L-attributed SDT (i.e., suitable for implementation). **(03-Marks)**

(iii) Consider, SDT for if-else statement

S -> if ({ L1=new(); L2=new(); C.false=L2; C.true=L1;} C)
{S_1.next=S.next;} S_1 else {S_2.next=S.next;} S_2
{S.code=C.code || **label** || L1 || S_1.code ||**label** ||L2|| S_2.code}

Let us turn this SDT(L-attributed) into an SDT that can operate with an LR parse (i.e., suitable during bottom-up parsing).**(03-Marks)** (10.0 Marks)

3.b. (i) Generate Three-address code for the following code **(04-Marks)**

```
switch(ch)
{
  case 1 : c = a + b;
           break;
  case 2 : c = a – b;
           break;
  default: …
}
```

(ii) Construct control flow graph (CFG) for the given three-address code (TAC) and then convert the code in CFG to SSA form. **(06-Marks)**

(10.0 Marks)

```
    i = 1
    j = 1
    k = 0
L1: ifFalse k < 100 goto L3
    ifFalse j < 20 goto L2
    j = i
    k = k + 1
    goto L1
L2: j = k
    k = k + 1
    goto L1
L3: return j
```

3.c. **(i)** In the below code, reduce the loop overhead from 100% to 50% (i.e, make only 50 iterations, instead of 100, afterwards, only 50% of the jumps and conditional branches in loop) (02-Marks)

```
for (i = 0; i < 100; i++)
  g();
```

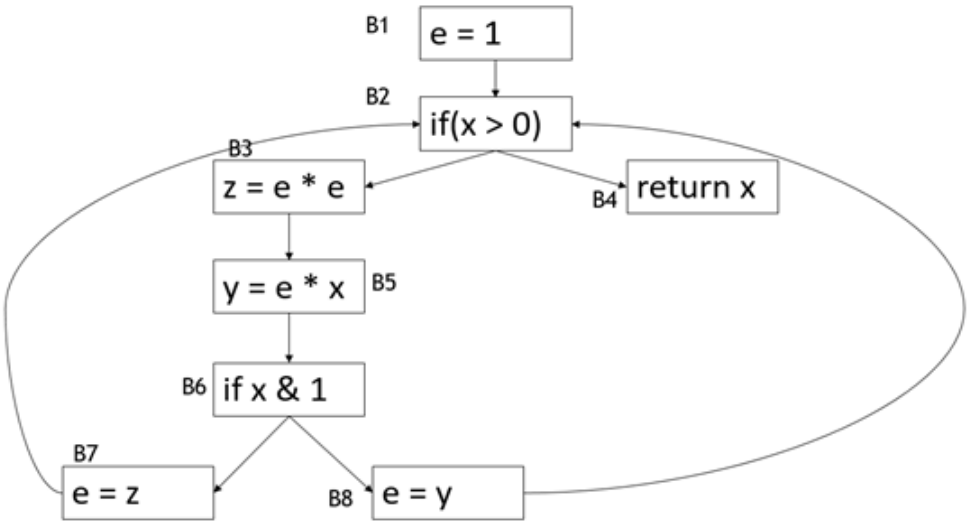
(ii) Apply the following optimization techniques to optimize the below given three address code **(03-Marks)**

- Loop strength reduction by induction Variables
- Constant folding and dead code elimination

```
   i = 0
L: i = i + 1
   t1 = 4 * i
   t2 = a[t1]
   if t2 < v goto L
```

(5.0 Marks)

4.a. i) Calculate use[B], def[B] and then find IN[B], OUT[B] by applying *Live-variable Analysis Algorithm* on the following flow graph. **(07-Marks)**



ii) Explain in brief any three issues in the design of a code generator. **(03-Marks)**

(10.0 Marks)

4.b. i) What is activation record? Construct an Activation Tree for the following Program. **[05-Marks]**

```
int main()
{
    int f5, f2;
    f5 = fib(5);
    f2 = fib(2);
}

int fib(int n)
{
    if(n<3)
        return 1;
    return fib(n-1) + fib(n-2);
}
```

ii) Explain in brief Calling sequence and Access link? **[04-Marks]** (9.0 Marks)

4.c. Generate target code sequence for the following basic block three address statements with register and address descriptors. **[6- Marks]**

```
x = y+z
z = x*x
y = z
x = y+z
```

Note 1: Assuming only two registers (R1 and R2) are available and all the variables are live on exit from the basic block. No temporary variables are used.
Note 2: Assuming that all arithmetic operations take their operands from registers.

(6.0 Marks)